attribute preserves the “dot” notation, `Input.Email`, so that model binding works correctly when the field is `POST`ed to the Razor Page.

Also, the initial `value` of the field has been set to the value currently stored in the property (`"test@example.com"`, in this case). The `type` of the element has also been set to the HTML5 `email` type, instead of using the default `text` type.

Perhaps the most striking addition is the swath of `data-val-*` attributes. These can be used by client-side JavaScript libraries such as jQuery to provide client-side validation of your `DataAnnotations` constraints. Client-side validation provides instant feedback to users when the values they enter are invalid, providing a smoother user experience than can be achieved with server-side validation alone, as I described in chapter 16.

Client-side validation

To enable client-side validation in your application, you need to add some jQuery libraries to your HTML pages. In particular, you need to include the jQuery, jQuery-validation, and jQuery-validation-unobtrusive JavaScript libraries. You can do this in several ways, but the simplest is to include the script files at the bottom of your view using

```
<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
<script src="~/lib/jquery-validation-unobtrusive/
➥ jquery.validate.unobtrusive.min.js"></script>
```

The default templates include these scripts for you in a handy partial template that you can add to your page in a `Scripts` section. If you’re using the default layout and need to add client-side validation to your view, add the following section somewhere on your view:

```
@section Scripts{
    @Html.Partial("_ValidationScriptsPartial")
}
```

This partial view references files in your wwwroot folder. The default layout template includes jQuery itself. If you don’t need to use jQuery in your application, you may want to consider a small